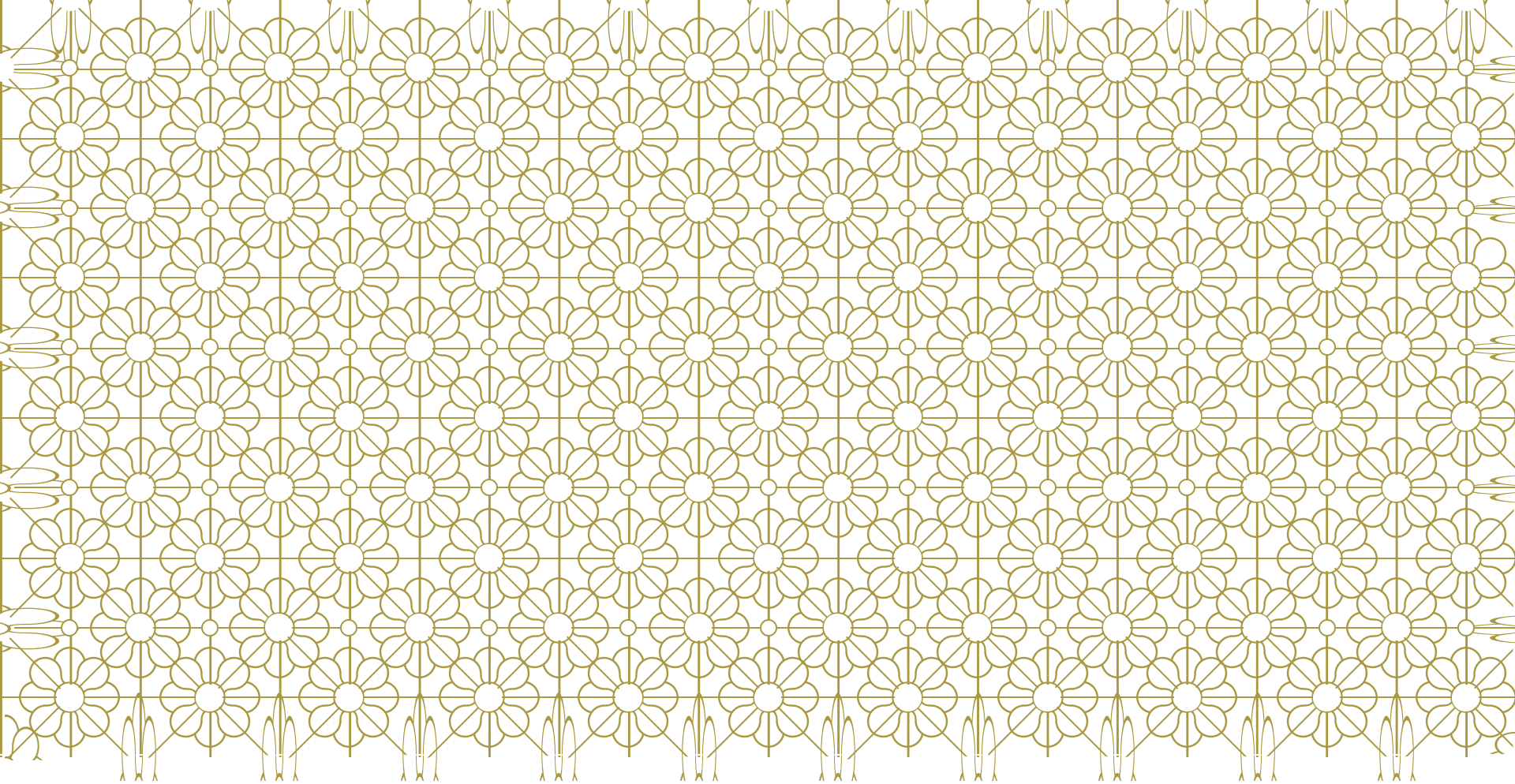
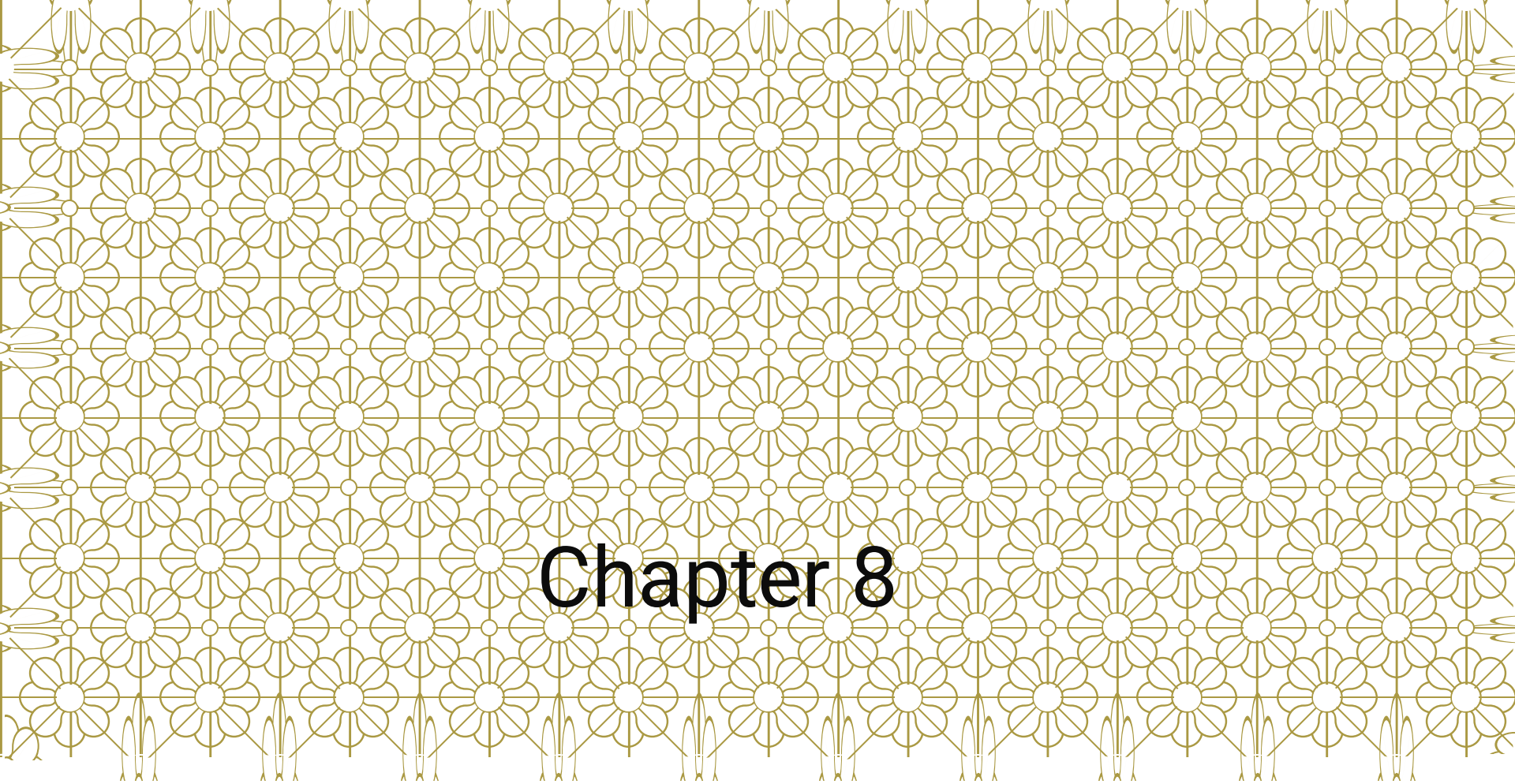




Algorithms analysis and Design

By : Dr. Basem
Alijla



Chapter 8

Dynamic Programming

OUTLINE

Change-making problem

Coin-collecting problem

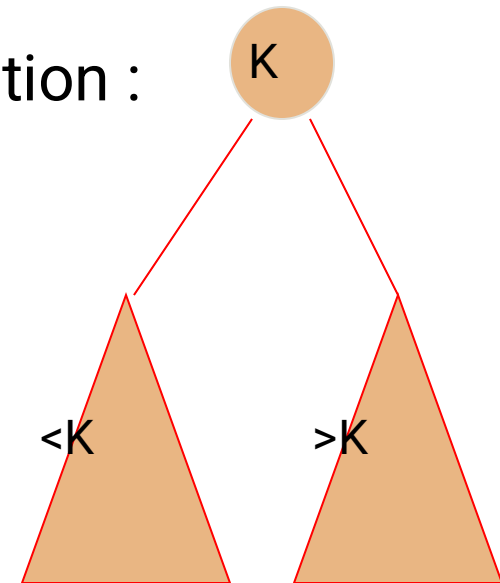
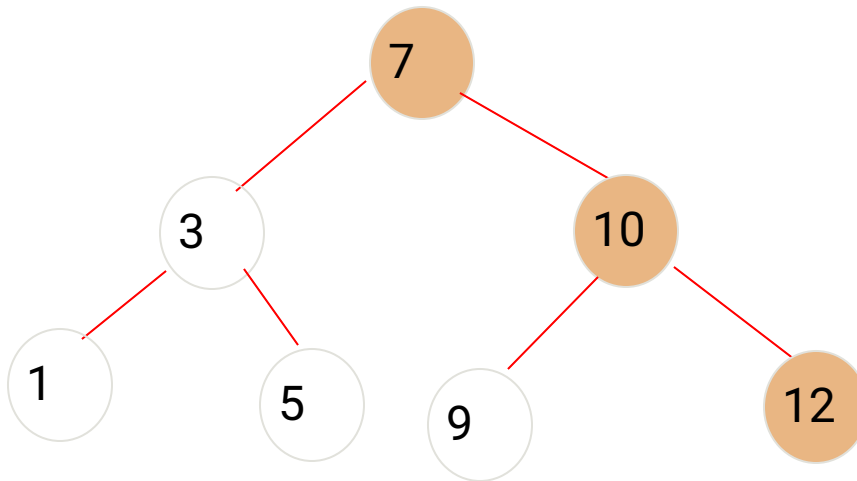
The Knapsack Problem and Memory Functions

Optimal Binary Search Tree

Warshall's and Floyd's Algorithms

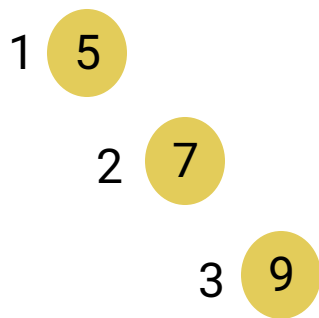
What is Binary Search Tree

- BST is a binary tree arrange keys in which left subtree has values less than the node's value and right subtree has values greater than node's value
- Utilizing Binary tree in search operation :
- set of Keys : 7, 3, 1, 10, 12, 5, 9



Balanced Binary Search Tree

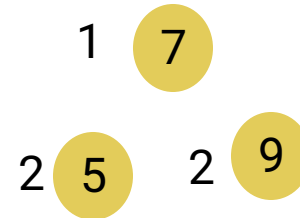
Keys : 5, 7, 9



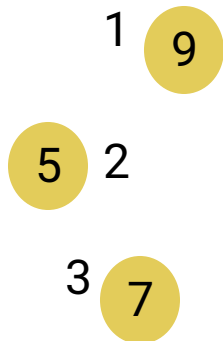
Average number
of Comparisons
 $= 1 + 2 + 3 = 6/3$



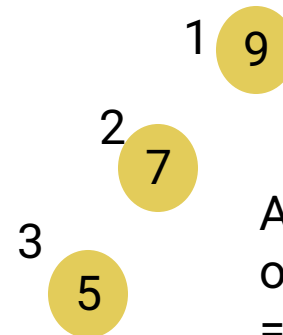
Average number
of Comparisons
 $= 1 + 2 + 3 = 6/3$



Average number
of Comparisons
 $= 1 + 2 + 3 = 5/3$



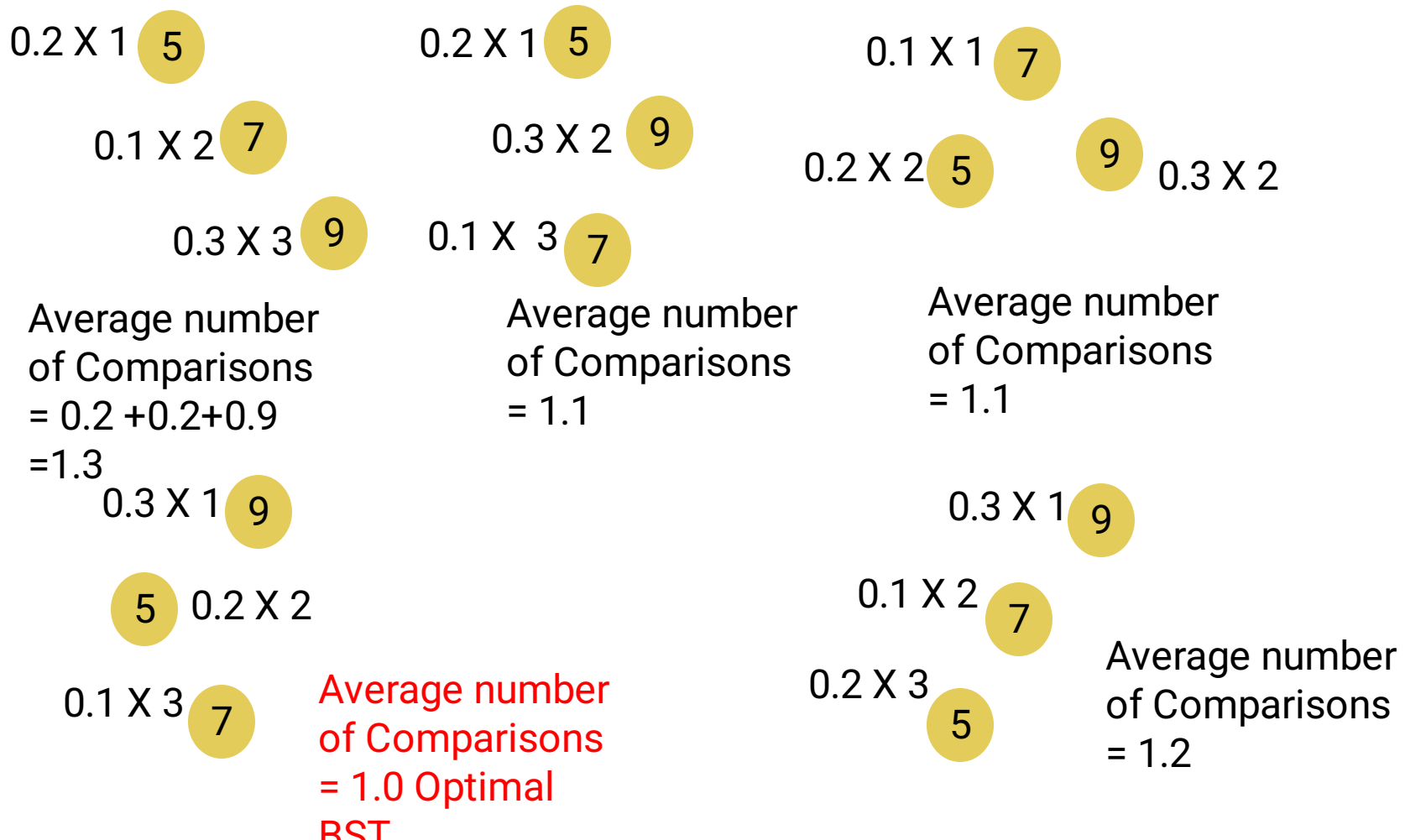
Average number
of Comparisons
 $= 1 + 2 + 3 = 6/3$



Average number
of Comparisons
 $= 1 + 2 + 3 = 6/3$

Optimal Binary Search Tree

Keys : 5, 7, 9 with propagability of search 0.2 , 0.1 , and 0.3



Optimal Binary Search Trees

Problem:

Given n keys $a_1 < \dots < a_n$ and probabilities $p_1 \leq \dots \leq p_n$ searching for them. find a BST with a minimum average number of comparisons in successful search.

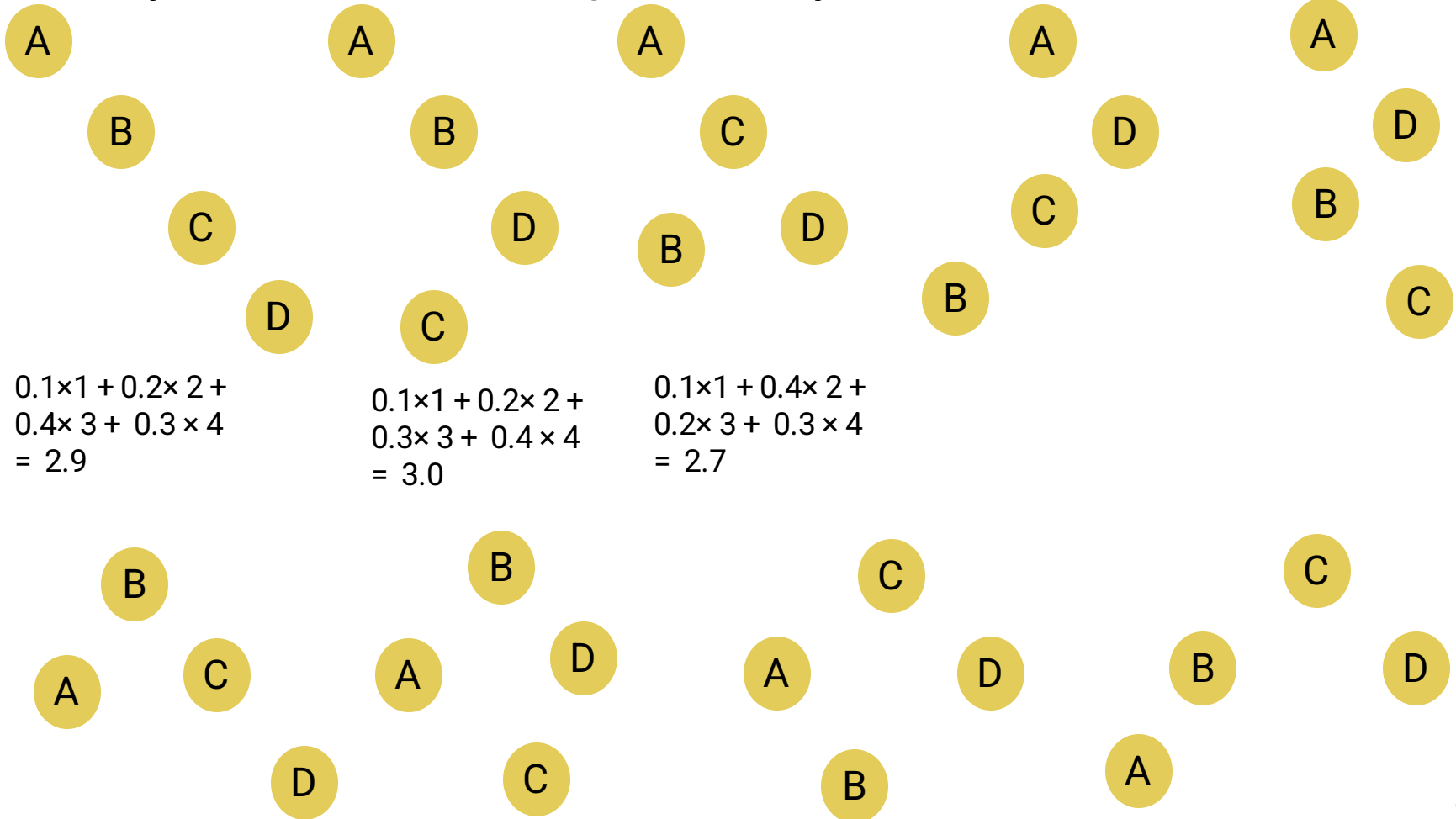
Since total number of BSTs with n nodes is given by n th Catalan number

$$c(n) = \frac{1}{n+1} \binom{2n}{n} \quad \text{for } n > 0, \quad c(0) = 1,$$

which grows exponentially, brute force is hopeless.

Total subtrees

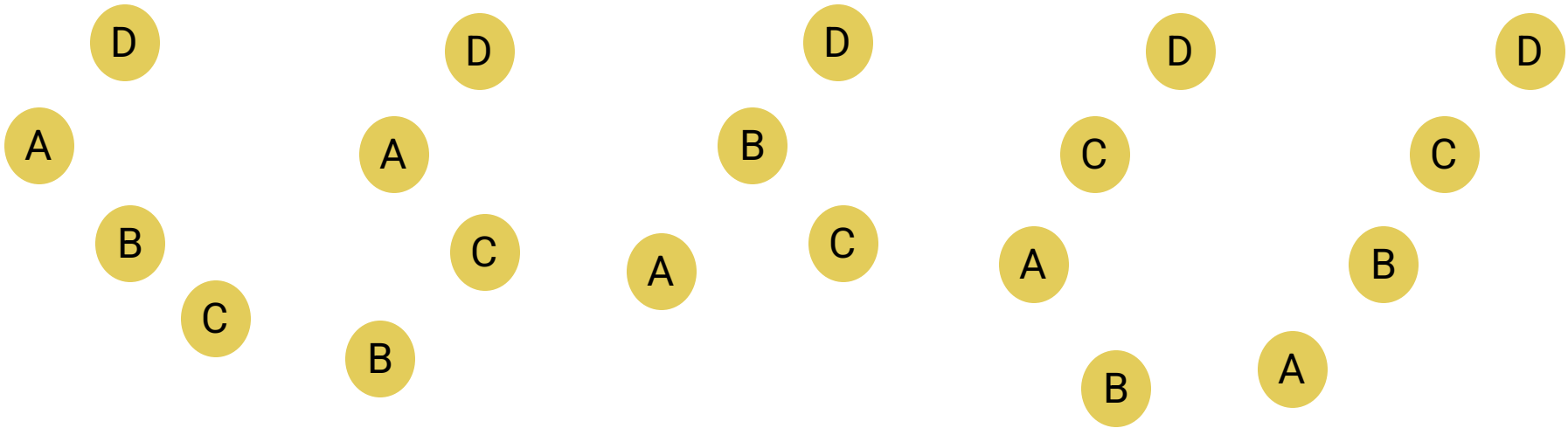
Keys: A, B, C, D, with probability 0.1, 0.2, 0.4, 0.3



Total subtrees

Keys: A, B, C, D, with probability 0.1, 0.2, 0.4, 0.3

14 subtrees



$$\begin{aligned} &0.4 \times 1 + 0.1 \times 2 + \\ &0.2 \times 3 + 0.3 \times 4 \\ &= 2.4 \end{aligned}$$

Optimal Binary Search Trees

- ❖ let $a_1, \dots, a_n$ be distinct keys ordered from the smallest to the largest and let $p_1, \dots, p_n$ be the probabilities of searching for them
- ❖ Let $C(i,j)$ be the smallest average number of comparisons made in a successful search in a binary search tree T_i^j made up of keys $a_i, \dots, a_j$, where i, j are some integer indices, $1 \leq i \leq j \leq n$.

Optimal Binary Search Trees

- we will find values of $C(i, j)$ for all smaller instances of the problem . Although we are interested just in $C(1, n)$.
- will consider all possible ways to choose a root a_k among the keys $a_i, \dots, a_j$.

Optimal Binary Search Trees

- Consider the Keys : 5, 10, 20, 15 with propagability of search 0.2 , 0.1 , 0.3 and 0.4

1	2	3	4
5	10	15	20
0.2	0.1	0.4	0.3

- Consider the tree is Empty, then the average number of comparison is zero

Optimal Binary Search Trees

1	2	3	4
5	10	15	20
0.2	0.1	0.4	0.3

Cases: Empty trees , cost = 0

$$C(1, 0) , C(2, 1) , C(3, 2) , C(4, 3) , C(5, 4) = 0$$

	0	1	2	3	4
1	0				
2		0			
3			0		
4				0	
5					0

C

	0	1	2	3	4
1		1			
2			2		
3				3	
4					4
5					

R

Optimal Binary Search Trees

Case: Trees with one key

1	2	3	4
5	10	15	20
0.2	0.1	0.4	0.3



Optimal Binary Search Trees

1	2	3	4
5	10	15	20
0.2	0.1	0.4	0.3

Cases: Trees with one key, cost = P_k

$$C(1, 1) = 0.2, C(2, 2) = 0.1$$

$$C(3, 3) = 0.4, C(4, 4) = 0.3$$

	0	1	2	3	4
1	0	0.2			
2		0	0.1		
3			0	0.4	
4				0	0.3
5					0

C

	0	1	2	3	4
1		1			
2			2		
3				3	
4					4
5					

R

Optimal Binary Search Trees

Cases: Trees with two keys

1	2	3	4
5	10	15	20
0.2	0.1	0.4	0.3

$C(1, 2)$, $C(2, 3)$, $C(3,4)$

	0	1	2	3	4
1	0	0.2	$C(1,2)$		
2		0	0.1	$C(2,3)$	
3			0	0.4	$C(3,4)$
4				0	0.3
5					0

C

	0	1	2	3	4
1		1			
2			2		
3				3	
4					4
5					

R

Optimal Binary Search Trees

Case: Trees with two keys

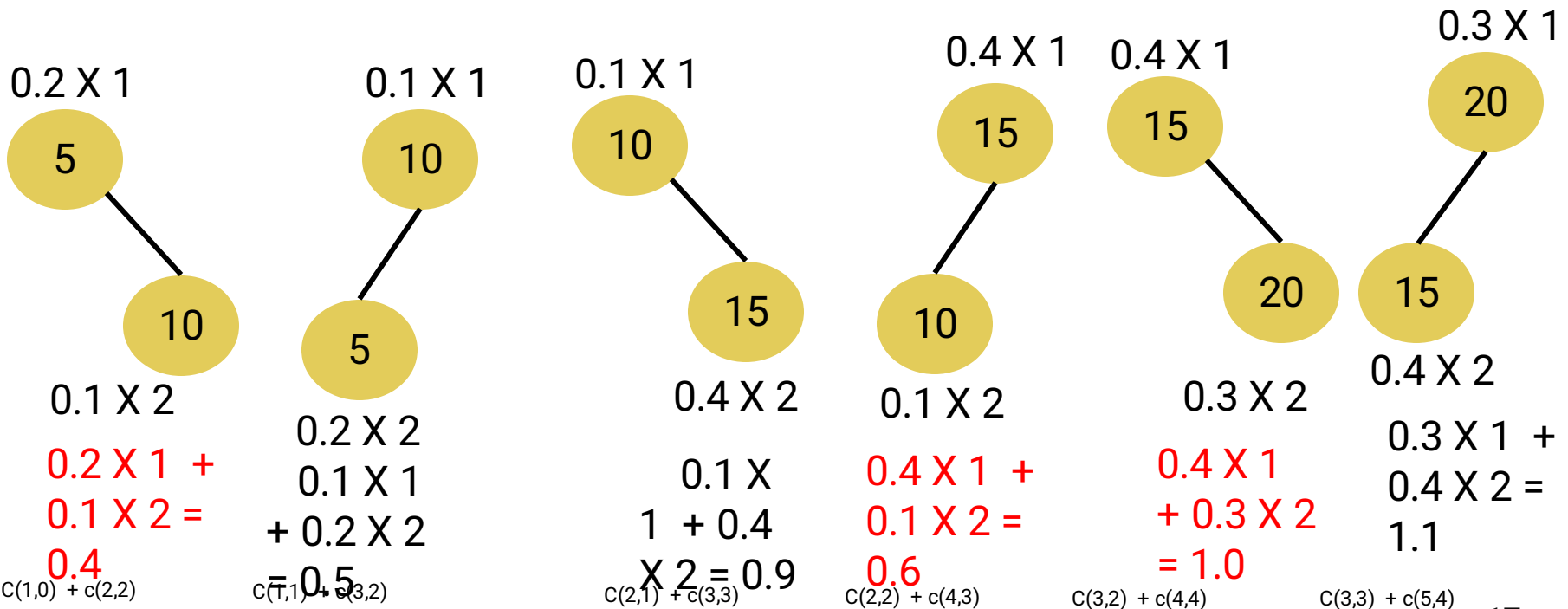
1	2	3	4
5	10	15	20
0.2	0.1	0.4	0.3

$$C(1,2) = 0.4$$

$$C(2,3) = 0.6$$

$$C(3,4) = 1.0$$

$$\min_{i \leq k \leq j} C(i, k)$$



Optimal Binary Search Trees

Cases: Trees with two keys

1	2	3	4
5	10	15	20
0.2	0.1	0.4	0.3

$C(1, 2)$, $C(2, 3)$, $C(3,4)$

	0	1	2	3	4
1	0	0.2	0.4		
2		0	0.1	0.6	
3			0	0.4	1.0
4				0	0.3
5					0

C

	0	1	2	3	4
1		1	1		
2			2	3	
3				3	3
4					4
5					

R

Optimal Binary Search Trees

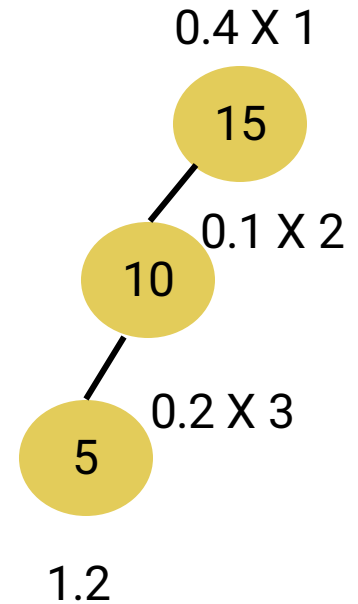
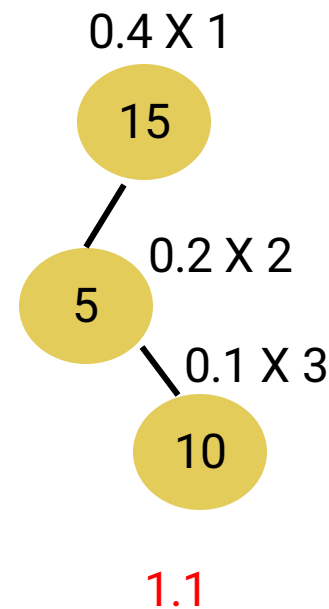
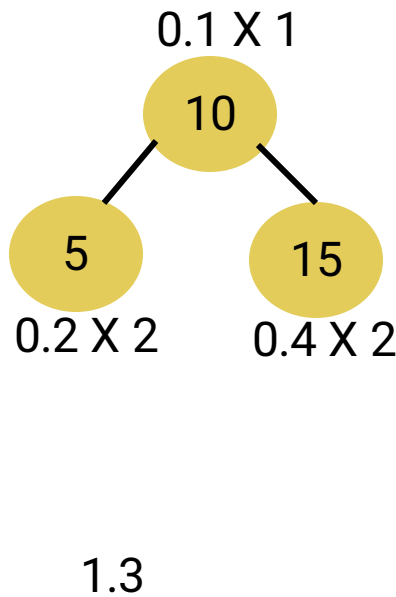
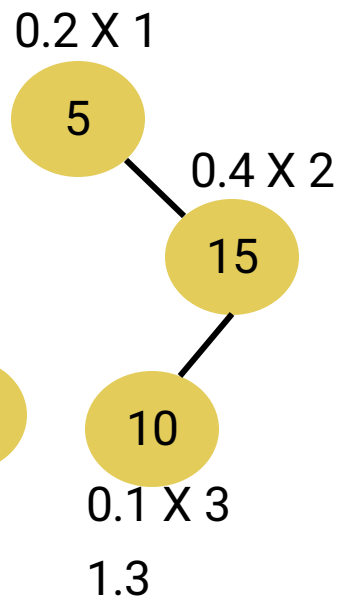
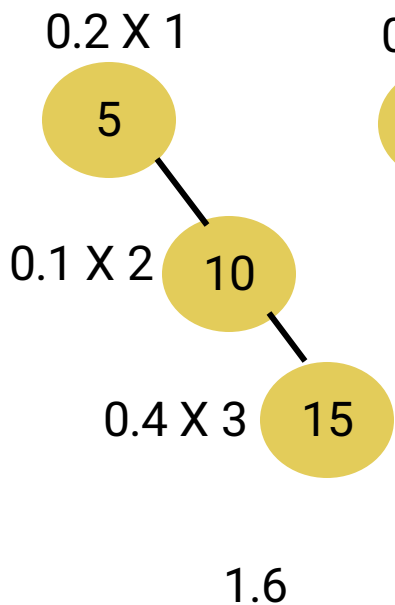
Case: Trees with two keys

$$C(1,3) = 1.1$$

$$C(2,4) = 1.2$$

$$\min_{i \leq k \leq j} C(i, l)$$

1	2	3	4
5	10	15	20
0.2	0.1	0.4	0.3



Optimal Binary Search Trees

Cases: Trees with three keys

1	2	3	4
10	40	15	20
0.2	0.1	0.4	0.3

$$C(1, 3) = \min \{C(1,0)+C(2,3), C(1,1)+C(3,3), C(1,2)+C(4,3)\} + P_s = 1.1$$

$$C(2, 4) = \min\{C(2,1)+C(3,4), C(2,2)+C(4,4), C(2,3)+C(5,4))\}+PS = 1.2$$

	0	1	2	3	4
1	0	0.2	0.4	1.1	
2		0	0.1	0.6	1.2
3			0	0.4	1.0
4				0	0.3
5					0

C

	0	1	2	3	4
1		1	1	3	3
2			2	3	3
3				3	3
4					4
5					

R

Optimal Binary Search Trees

Cases: Trees with three keys

1	2	3	4
5	10	15	20
0.2	0.1	0.4	0.3

$$C(1, 3) = \min \{C(1,0)+C(2,4), C(1,1)+C(3,34), C(1,2)+C(4,4), C(1,3)+C(5,4)\} + P_s = 1.7$$

	0	1	2	3	4
1	0	0.2	0.4	1.1	1.7
2		0	0.1	0.6	1.2
3			0	0.4	1.0
4				0	0.3
5					0

C

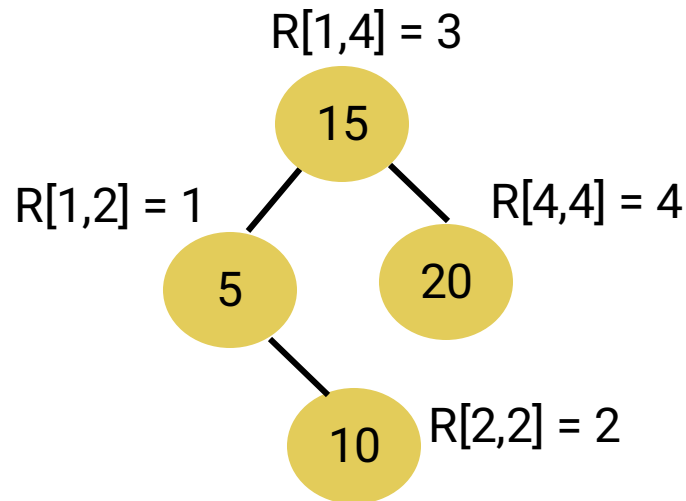
	0	1	2	3	4
1		1	1	3	3
2			2	3	3
3				3	3
4					4
5					

R

Optimal Binary Search Trees

1	2	3	4
5	10	15	20
0.2	0.1	0.4	0.3

Optimal BST



	0	1	2	3	4
1		1	1	3	3
2			2	3	3
3				3	3
4					4
5					

R

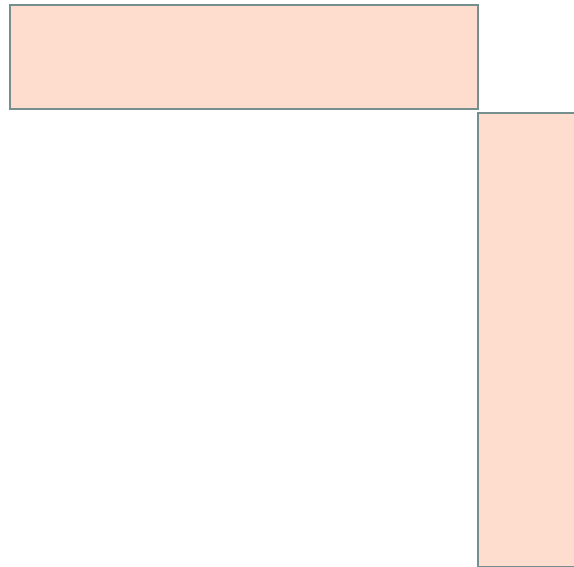
Optimal Binary Search Trees

$$\begin{aligned}
 \diamond C(i, j) &= \min_{i \leq k \leq j} \left\{ p_k * 1 + \sum_{s=i}^{k-1} p_s * (\text{level of } a_s \text{ in } T_i^{k-1} \right. \\
 &\quad \left. + 1) + \sum_{s=k+1}^j p_s * (\text{level of } a_s \text{ in } T_{k+1}^j + 1) \right\} \\
 \diamond &= \min_{i \leq k \leq j} \left\{ \sum_{s=i}^{k-1} p_s * \text{level of } a_s \text{ in } T_i^{k-1} \right. \\
 &\quad \left. + \sum_{s=k+1}^j p_s * \text{level of } a_s \text{ in } T_{k+1}^j + \sum_{s=i}^j p_s \right\}
 \end{aligned}$$

Optimal Binary Search Trees

- ❖ $= \min_{i \leq k \leq j} \left\{ C(i, k-1) + C(k+1, j) + \sum_{s=i}^j p_s \right\}$
- ❖ $C(i, j) = \min_{i \leq k \leq j} \left\{ C(i, k-1) + C(k+1, j) + \sum_{s=i}^j p_s \right\}$ for $1 \leq i \leq j \leq n$
- ❖ The number of comparisons in **the empty tree**
- ❖ $C(i, i-1) = 0$ for $1 \leq i \leq n+1$, // *empty tree*
- ❖ $C(i, i) = p_i$ for $1 \leq i \leq n$ // *only one node tree*

Optimal Binary Search Trees



Example:

key A B C D
probability 0.1 0.2 0.4 0.3

	0	1A	2B	3C	4D
1A	0	.1	C(1,2)	C(1,3)	C(1,4)
2B		0	.2	C(2,3)	C(2,4)
3C			0	.4	C(3,4)
4D				0	.3
5					0

C

	0	1A	2B	3C	4D
1A		1			
2B			2		
3C				3	
4D					4
5					

R

$R(i, i) = i$ for $1 \leq i \leq n$.

indices of the roots of the optimal subtree

$$C(1,2) = \min \{ C[i,k-1] + C[k+1,j] \}$$

$i = 1, j = 2, k = 1, 2$
 $\min(c(1,0)+c(2,2), c(1,1)+c(3,2))+P_s = 0.4, k = 2$

	0	1A	2B	3C	4D
1A	0	.1	0.4	$C(1,3)$	$C(1,4)$
2B	.2	0	.2	$C(2,3)$	$C(2,4)$
3C		.1	0	.4	$C(3,4)$
4D				0	.3
5					0

	0	1A	2B	3C	4D
1A		1	2		
2B			2		
3C				3	
4D					4
5					

$$C(1, 2) = \min \left\{ \begin{array}{l} k = 1: C(1, 0) + C(2, 2) + \sum_{s=1}^2 p_s = 0 + 0.2 + 0.3 = 0.5 \\ k = 2: C(1, 1) + C(3, 2) + \sum_{s=1}^2 p_s = 0.1 + 0 + 0.3 = 0.4 \end{array} \right\}$$

$= 0.4.$

$$C[i,k-1] + C[k+1,j]\}$$

$C(2,3)$

$i = 2, j = 3, k = 2, 3 \min(c(2,1)+c(3,3), c(2,2)+c(4,3))+P_s = 0.8, k = 3$

	0	1A	2B	3C	4D
1A	0	.1	0.4	$C(1,3)$	$C(1,4)$
2B		0	.2	0.8	$C(2,4)$
3C		.4	0	.4	$C(3,4)$
4D			.2	0	.3
5					0

The diagram shows the calculation of $C(2,3)$ using the dynamic programming table. The value 0.8 is highlighted in blue in the cell (2,3). Arrows indicate the path taken to reach this value: a vertical arrow from (2,1) to (2,2) and a horizontal arrow from (2,2) to (2,3). The values .4 and .2 are highlighted in orange in the cells (3,1) and (4,1) respectively, representing the costs of the subproblems.

	0	1A	2B	3C	4D
1A		1	2		
2B			2	3	
3C				3	
4D					4
5					

$$C[i,k-1] + C[k+1,j]\}$$

$C(3,4)$

$i = 3, j = 4, k = 3, 4$ $\min(c(3,2)+c(4,4), c(3,3)+c(5,4))+P_s = 1.0$,
 $k = 3$

	0	1A	2B	3C	4D
1A	0	.1	0.4	$C(1,3)$	$C(1,4)$
2B		0	.2	0.8	$C(2,4)$
3C			0	.4	1.0
4D			.3	0	.3
5				.4	0

The diagram illustrates the calculation of $C(3,4)$. It shows a grid with rows 1A, 2B, 3C, 4D, and 5, and columns 0, 1A, 2B, 3C, and 4D. The value 1.0 is highlighted in the cell (3C, 4D). Arrows indicate the path from (3,2) to (3,3) and (3,3) to (3,4).

	0	1A	2B	3C	4D
1A		1	2		
2B			2	3	
3C				3	3
4D					4
5					

The diagram illustrates the calculation of $C(3,4)$. It shows a grid with rows 1A, 2B, 3C, 4D, and 5, and columns 0, 1A, 2B, 3C, and 4D. The value 3 is highlighted in the cell (3C, 4D). Arrows indicate the path from (3,2) to (3,3) and (3,3) to (3,4).

$$C[i,k-1] + C[k+1,j]\}$$

$C(1,3)$

$i=1, j=3, k=1,2,3 \min(c(1,0)+c(2,3), c(1,1)+c(3,3), c(1,2)+c(4,3))+Ps = 1.1, k=3$

	0	1A	2B	3C	4D
1A	0	.1	0.4	1.1	$C(1,4)$
2B	.8	0	.2	0.8	$C(2,4)$
3C		.5	0	.4	1.0
4D			.4	0	.3
5					0

	0	1A	2B	3C	4D
1A		1	2	3	
2B			2	3	
3C				3	3
4D					4
5					

$$C[i,k-1] + C[k+1,j]\}$$

$C(2,4)$

$i = 2, j = 4, k = 2, 3, 4$ $\min(c(2,1)+c(3,4), c(2,2)+c(4,4), c(2,3)+c(5,4)) + P_s = 1.4, k = 3$

	0	1A	2B	3C	4D
1A	0	.1	0.4	1.1	$C(1,4)$
2B		0	.2	0.8	1.4
3C		1.0	0	.4	1.0
4D			.5	0	.3
5				.8	0

	0	1A	2B	3C	4D
1A		1	2	3	
2B			2	3	3
3C				3	3
4D					4
5					

$$C[i,k-1] + C[k+1,j]\}$$

$C(1,4)$

$i=1, j=4, k=1, 2, 3, 4 \min(c(1,0)+C(2,4), c(1,1)+c(3,4), c(1,2)+c(4,4), c(1,3)+c(5,4))+P_s = 1.7, k=3$

	0	1A	2B	3C	4D
1A	0	.1	0.4	1.1	1.7
2B	1.4	0	.2	0.8	1.4
3C		1.1	0	.4	1.0
4D			.7	0	.3
5				1.1	0

	0	1A	2B	3C	4D
1A		1	2	3	3
2B			2	3	3
3C				3	3
4D					4
5					

The optimal Subtree :

The optimal Subtree :

The average number of key comparisons in the optimal tree is equal to 1.7 .

$$\begin{aligned} &0.4 \times 1 + 0.2 \times 2 + \\ &0.3 \times 2 + 0.1 \times 3 \\ &= 1.7 \end{aligned}$$

